Documentação

➔ Identificação do Projeto

- **Nome do Jogo:** Jogo da Velha (Jogador vs. Computador)
- **Disciplina:** Linguagem de Programação
- **Data de Entrega:** 16/10/2026
- **Integrantes do Grupo:**
    - Maria Júlia Gadelha
    - Lucas Mourão

➔ Descrição do funcionamento

O projeto consiste em uma implementação em console (terminal) do clássico **Jogo da Velha**, desenvolvida inteiramente em linguagem C. O programa coloca um jogador humano (X) para disputar uma partida contra o computador (O), que realiza suas jogadas de forma automatizada utilizando números aleatórios.

O fluxo do jogo é controlado por um laço de repetição principal (while), que alterna os turnos entre o usuário e a máquina a cada rodada, atualizando o tabuleiro dinamicamente e verificando em tempo real se houve vitória, empate ou se a partida deve continuar.

➔ Regras

1. O tabuleiro possui uma matriz de dimensões **3x3**.
2. O jogador humano inicia jogando com o símbolo **'X'**, e o computador joga com o símbolo **'O'**.
3. O jogador escolhe uma posição informando a **linha** (1 a 3) e a **coluna** (1 a 3).
4. O computador seleciona sua jogada de forma aleatória utilizando a função rand().
5. **Validações:** Não é permitida a escolha de posições fora dos limites do tabuleiro ou em locais que já estejam ocupados.
6. **Condição de Vitória:** O primeiro a alinhar 3 símbolos iguais em linha, coluna ou diagonal vence a partida.

7. **Condição de Empate:** Se todas as 9 posições forem preenchidas sem que haja um vencedor, o jogo termina empatado.

➔ Explicação do código

1. Bibliotecas e Variáveis Globais

O código começa incluindo as bibliotecas necessárias e definindo variáveis que serão acessadas por várias funções do programa:

1. **#include <stdio.h>**: Biblioteca padrão para entrada e saída de dados (como printf e scanf).
2. **#include <windows.h>**: Usada para funções do sistema operacional Windows, como Sleep() (pausa) e system("cls") (limpar tela).
3. **#include <stdbool.h>**: Permite o uso do tipo booleano (true e false).
4. **#include <time.h>**: Usada para gerar números aleatórios baseados no horário atual (srand(time(NULL))).

```
#include <stdio.h>
#include <windows.h>
#include <stdbool.h>
#include <time.h>
```

Variáveis Globais Principais:

1. **char tabuleiro[3][3]**: Matriz que representa o tabuleiro do jogo da velha (3 linhas por 3 colunas).
2. **bool parar = false**: Controla quando o jogo deve terminar.
3. **char rodada = 'X'**: Define de quem é a vez (começa com o jogador X).
4. **int jogadas = 0:** Conta o número total de jogadas para detectar empates (máximo = 9).
5. **int tempoPc e tempoLeitura**: Controlam o tempo de espera (em milissegundos) para mensagens e jogadas do computador.

```
char tabuleiro[3][3];
bool parar = false;
char rodada = 'X';
int jogadas = 0;
int linha = 0;
int coluna = 0;
int tempoPc = 1500;
int tempoLeitura = 1200;
```

2. Funções do Jogo

1. **iniciarTabuleiro()**: Preenche todas as posições da matriz tabuleiro com espaços em branco (' '), garantindo que o jogo comece limpo.

```c
void iniciarTabuleiro(){
    // Inicia o Tabuleiro com Espaços ' '
    for(int i = 0; i<3; i++){
        for(int j = 0; j<3; j++){
            tabuleiro[i][j] = ' ';
        }
    }
}
```

2. **mostrarTabuleiro():** Responsável por desenhar o tabuleiro formatado no console com as coordenadas visuais das linhas (1 a 3) e colunas (1 a 3), mostrando as peças X, O ou espaços vazios.

```c
void mostrarTabuleiro(){
    // Printa o Tabuleiro Organizado
    printf("\n   1   2   3\n");
    for(int i = 0; i < 3; i++){
        printf("%d ", i+1);
        for(int j = 0; j < 3; j++){
            printf("[%c] ", tabuleiro[i][j]);
        }
        printf("\n");
    }
}
```

3. **jogadaJogador()**:

- Controla a entrada de dados do usuário: Pede ao usuário para digitar a linha e a coluna.
- Proteção de entrada: Verifica se o usuário digitou realmente um número (evitando bugs caso digitem letras). Se houver erro, limpa o buffer do teclado com getchar() e aguarda.
- Ajusta os valores digitados (subtraindo 1) para corresponder aos índices da matriz em C (que vão de 0 a 2).
- Valida se a posição escolhida está dentro dos limites do tabuleiro e se já não está ocupada.

```c
bool jogadaJogador(){
    // Recebe e Valida a Linha e a Coluna
    printf("\n=> Seu turno (X)!\n");

    // Proteção para a linha
    printf("Insira a linha: ");
    if (scanf("%d", &linha) != 1) {
        printf("\n=== ENTRADA INVALIDA! DIGITE APENAS NUMEROS. ===\n");
        while (getchar() != '\n'); // Limpa o buffer de entrada
        Sleep(tempoLeitura);
        return false;
    }
    linha -= 1;

    // Proteção para a coluna
    printf("Insira a coluna: ");
    if (scanf("%d", &coluna) != 1) {
        printf("\n=== ENTRADA INVALIDA! DIGITE APENAS NUMEROS. ===\n");
        while (getchar() != '\n'); // Limpa o buffer de entrada
        Sleep(tempoLeitura);
        return false;
    }
    coluna -= 1;

    // Validação de limites
    if((linha < 0) || (linha > 2) || (coluna < 0) || (coluna > 2)){
        printf("\n=== POSICAO INVALIDA ===\n");
        Sleep(tempoLeitura);
        return false;
    }

    // Validação de posição ocupada
    if(tabuleiro[linha][coluna] != ' '){
        printf("\n=== POSICAO JA OCUPADA ===\n");
        Sleep(tempoLeitura);
        return false;
    }

    return true;
}
```

4. **jogadaMaquina()**:

- Controla a jogada do computador (O):
- Utiliza um laço do-while combinando rand() % 3 para gerar coordenadas aleatórias para a linha e a coluna até encontrar uma posição que esteja vazia (' ').
- Inclui uma pausa (Sleep) para simular que o computador está "pensando".

```c
void jogadaMaquina(){
    printf("\n=> Turno do Computador (O)...\n");
    Sleep(tempoPc);

    do{
        linha = rand() % 3;
        coluna = rand() % 3;
    } while(tabuleiro[linha][coluna] != ' ');
}
```

5. **validarVitoria(char r)**:

- Verifica se o jogador atual (r) venceu a partida:
- Checa todas as 3 linhas e 3 colunas.
- Checa as duas diagonais principais.
- Se encontrar uma combinação vencedora, altera a variável parar para true e exibe a mensagem de vitória correspondente.

```c
void validarVitoria(char r){
    for (int i = 0; i < 3; i++) {
        if (tabuleiro[i][0] == r && tabuleiro[i][1] == r && tabuleiro[i][2] == r) parar = true;
        if (tabuleiro[0][i] == r && tabuleiro[1][i] == r && tabuleiro[2][i] == r) parar = true;
    }
    if (tabuleiro[0][0] == r && tabuleiro[1][1] == r && tabuleiro[2][2] == r) parar = true;
    if (tabuleiro[0][2] == r && tabuleiro[1][1] == r && tabuleiro[2][0] == r) parar = true;

    if (parar) {
        system("cls"); // Limpa para mostrar o tabuleiro final
        printf("*** JOGO DA VELHA ***\n");
        mostrarTabuleiro();
        if (r == 'X') {
            printf("\n==== VOCE (X) VENCEU! ====\n");
        } else {
            printf("\n==== O COMPUTADOR (O) VENCEU! ====\n");
        }
    }
}
```

3. Função Principal (main)

A parte principal do programa roda dentro de um laço **while(!parar)**:

- Limpa a tela (system("cls")) e exibe o tabuleiro atualizado.
- Verifica de quem é a vez:
- Se for o jogador (X), chama jogadaJogador(). Se a jogada for inválida, usa continue para reiniciar o turno.
- Se for a máquina (O), chama jogadaMaquina().
- Atualiza a matriz com a jogada e incrementa o contador de jogadas.

- Chama validarVitoria() para ver se alguém ganhou.
- Se o número de jogadas chegar a 9 sem vencedor, declara empate e encerra o jogo.
- Alterna a rodada (rodada = (rodada == 'X') ? 'O' : 'X').

```c
int main(){
    SetConsoleOutputCP(CP_UTF8);
    iniciarTabuleiro();
    srand(time(NULL));

    while(!parar){
        system("cls");

        printf("*** JOGO DA VELHA ***\n");
        mostrarTabuleiro();

        if(rodada == 'X'){
            // Se a jogada for inválida, reinicia o loop
            if(!jogadaJogador()){
                continue;
            }
        }
        else {
            jogadaMaquina();
        }

        tabuleiro[linha][coluna] = rodada;
        jogadas++;

        validarVitoria(rodada);

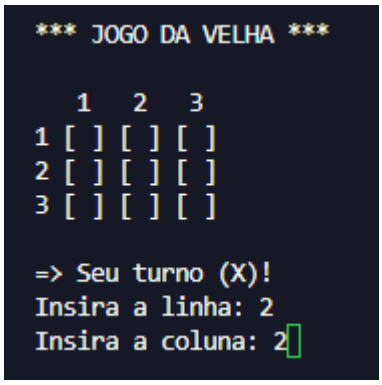
        if(parar){
            break;
        }

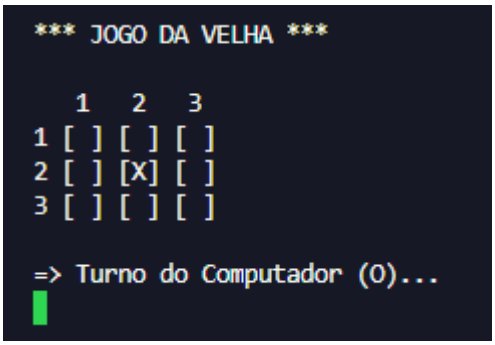
        if(jogadas == 9){
            system("cls");
            printf("*** JOGO DA VELHA ***\n");
            mostrarTabuleiro();
            printf("\n==== EMPATE! ====\n");
            break;
        }

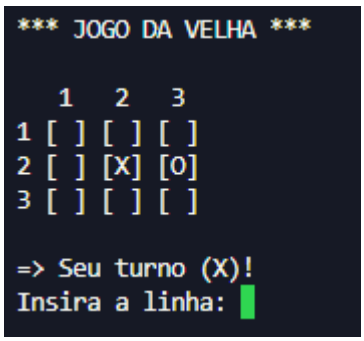
        rodada = (rodada == 'X') ? 'O' : 'X';
    }

    return 0;
}
```

➔ Exemplo de execução







➔ Dificuldades encontradas

- **Dificuldade 1 (Tratamento de entradas inválidas):** Inicialmente, caso o usuário digitasse uma letra em vez de um número no scanf, o programa entrava em loop infinito.
    - **Solução:** Implementou-se uma verificação do retorno do scanf combinada com while (getchar() != '\n'); para limpar o buffer de entrada e exibir uma mensagem ao usuário.
- **Dificuldade 2 (Jogadas aleatórias repetidas da máquina):** Garantir que o computador não sobrescrevesse jogadas do usuário ou jogasse em posições já ocupadas.

- **Solução:** Utilizou-se um laço do-while atrelado à validação do tabuleiro, forçando o sorteio de novas coordenadas até encontrar um espaço realmente vazio.